



Demystifying JavaScript Hoisting

Understanding the Mechanism and Impact on Code Execution

Hoisting in JavaScript: A Deep Dive

Hoisting is one of the most misunderstood concepts in JavaScript, often leading to confusion among developers. This guide aims to clarify what hoisting is and how it affects the execution of JavaScript code.

Simple Definition

Hoisting refers to the JavaScript mechanism where variables and function declarations are moved to the top of their containing scope during the compile phase. This process involves two main phases:

Phase 1 → Memory Creation

Phase 2 → Code Execution

Before JavaScript executes the code, it first prepares memory for variables and functions.

Real-Life Analogy

Consider a school classroom scenario:

Before students arrive, the teacher prepares an attendance sheet with their names:

Rahul

Aman

Priya

Although the students are not present yet, their names are already recorded. This is analogous to hoisting, where the identifiers are available before execution.

JavaScript Execution Phases

Consider the following JavaScript code:

```
var name = "Rahul";

function hello() {
  console.log("Hello");
}
```

Phase 1: Memory Creation

During this phase, JavaScript sets aside memory for variables and functions:

```
name = undefined
hello = complete function
```

Phase 2: Execution

In this phase, actual values are assigned:

```
name = Rahul
```

Example 1: var Hoisting

```
console.log(name);
```

```
var name = "Rahul";
```

Output:

```
undefined
```

Explanation

Internally, JavaScript processes this as:

```
var name;
```

```
console.log(name);
```

```
name = "Rahul";
```

During memory creation, name is initialized to undefined.

Example 2: Function Hoisting

```
hello();
```

```
function hello() {  
  console.log("Hello World");  
}
```

Output:

Hello World

Explanation

The function hello is fully hoisted, meaning it is available in memory as a complete function before any code is executed.

Example 3: let Hoisting

```
console.log(age);
```

```
let age = 22;
```

Output:

ReferenceError

Explanation

Many assume let is not hoisted, but it is. However, it remains in the **Temporal Dead Zone (TDZ)** until initialized.

TDZ Example

```
console.log(age);
```

```
let age = 22;
```

During memory creation:

```
age = <uninitialized>
```

Example 4: const Hoisting

```
console.log(city);
```

```
const city = "Delhi";
```

Output:

ReferenceError

Explanation

const variables also reside in the TDZ and must be initialized before use.

Difference Between var, let, and const

Keyword	Hoisted	Initial Value
var	Yes	undefined
let	Yes	Uninitialized
const	Yes	Uninitialized

Function Expression Hoisting

```
sayHi();
```

```
var sayHi = function() {  
  console.log("Hi");  
};
```

Output:

TypeError

Explanation

During memory creation:

```
sayHi = undefined
```

Hence, sayHi() results in undefined(), causing a TypeError.

Arrow Function Hoisting

```
hello();
```

```
const hello = () => {
```

```
console.log("Hello");  
};
```

Output:

ReferenceError

Explanation

As const is involved, it is affected by the TDZ.

Real-Life Company Analogy

Before a workday begins, a company prepares an employee list:

Employee List Created

Employee A

Employee B

Employee C

This is akin to the **Memory Creation Phase**. Once work starts, employees are assigned tasks:

Employee A → Coding

Employee B → Design

Employee C → Testing

This represents the **Execution Phase**.

Complete Example

```
console.log(a);
```

```
var a = 10;
```

```
console.log(a);
```

Creation Phase

a = undefined

Execution

- First console.log(a); outputs undefined.
- Then a is assigned a value of 10.
- The final console.log(a); outputs 10.

Interview Insight: Is JavaScript Interpreted?

JavaScript is partly interpreted. It first creates an **Execution Context**, performing:

1. Memory Creation Phase
2. Execution Phase

Hoisting occurs during the Memory Creation Phase.

Memory Trick for Hoisting

var

var name;

name = "Rahul";

Hoisted with:

undefined

let and const

Hoisted

BUT

TDZ

function

Fully Hoisted

Visual Flow of JavaScript Execution

JavaScript Starts

|

v

Create Global Execution Context

|

v

Memory Creation Phase

|

|-- var → undefined

|-- let → TDZ

|-- const → TDZ

|-- function → Complete Function

|

v

Execution Phase

|

|-- Assign Values

|-- Execute Code

|

v

Output

Final One-Line Definition

Hoisting is JavaScript's behavior of allocating memory for variables and functions before the code is executed. This is why:

var a;

becomes:

a = undefined

and

function hello(){}

is available even before its declaration appears in the code.